CalTools: Advanced Calibration Tools

Contents

0.1 Installation and help . . . 1
0.2 1. Calibration and distance choices . . . 2
0.3 2. Negative totals are supported . . . 3
0.4 3. Negative or zero starting weights require an explicit metric . . . 4
0.5 4. Find and explain incompatible totals . . . 4
0.6 5. Initial screening and bounded feasibility . . . 6
0.7 6. Projection algorithms: scope and limitations . . . 7
0.8 7. Relax bounds along a declared direction . . . 7
0.9 8. Modify margins explicitly . . . 8
0.10 9. Automatic and interactive workflows . . . 10
0.11 10. Random inclusion probabilities . . . 12
0.12 11. Reproducibility and limits . . . 13

CalTools 0.2.1 provides bounded calibration, explicit inconsistency diagnostics, feasibility checks, directional bound relaxation and auditable margin aggregation. All public identifiers, examples, messages, code comments and documentation are in English. **Version 0.2.0 changes the public API**; see the migration guide before updating existing scripts.

0.1 Installation and help

R >= 4.1.0 is required. Runtime dependencies are `lpSolve` and `quadprog`. There is no package-specific C++ compilation and no external `sourceCpp()` path.

```
install.packages(c("lpSolve", "quadprog"))
install.packages("CalTools_0.2.1.tar.gz", repos = NULL, type = "source")
library(CalTools)
?calibrate_weights
?analyze_inconsistency
?calibration_preflight
example("calibrate_weights", package = "CalTools")
```

The project includes roxygen2 source comments and generated `man/*.Rd` help files. The help pages work immediately after installation; users do not need roxygen2. Developers can run `roxygen2::roxygenise("CalTools")` to regenerate them. The maintainer email in `DESCRIPTION` is a declared placeholder; replace it before publication. See PUBLISHING.md.

0.2 1. Calibration and distance choices

For an $n \times p$ matrix X and a vector of known totals t, calibration seeks weights w satisfying $X^\top w = t$. Quadratic calibration minimizes

$$Q(w) = \frac{1}{2} \sum_i \frac{(w_i - d_i)^2}{s_i q_i}, \qquad s_i > 0,\ q_i > 0.$$

Here `d` contains starting weights, `q` contains distance factors and `metric_scale` represents s. The default $s_i = d_i$ requires positive initial weights and gives the usual chi-square calibration distance. Setting `metric_scale=1` selects a different, weighted Euclidean distance.

```
X <- cbind(
  group_A = c(rep(1, 5), rep(0, 5)),
  group_B = c(rep(0, 5), rep(1, 5)),
  age = 1:10
)
totals <- c(group_A = 24, group_B = 26, age = 290)
d <- rep(5, 10)
q <- rep(1, 10)
```

```
linear_fit <- calibrate_weights(X, d, totals)
bounded_fit <- calibrate_weights(X, d, totals, method = "truncated",
  lower = rep(1, 10), upper = c(rep(9, 9), 10))
raking_fit <- calibrate_weights(X, d, totals, method = "raking")
logit_fit <- calibrate_weights(X, d, totals, method = "logit",
  lower = seq(0, 1, length.out = 10), upper = seq(9, 12, length.out = 10))
bounded_fit
#> CalTools: truncated calibration ( dual )
#> Status: optimal  | rank: 3  | max scaled residual: 2.73e-16
#> Weight range: 3.8 to 6.2
crossprod(X, bounded_fit$weights)
#>          [,1]
#> group_A   24
#> group_B   26
#> age      290
```

`lower` and `upper` always bound **final weights**, not multiplicative factors. Both scalar and observation-specific vector bounds are accepted.

Method	Domain and implementation
`linear`	Analytic quadratic solution in a reduced constraint space.
`truncated`	Convex quadratic calibration with reversible bound activity. `solver="dual"` uses the reduced dual; `solver="qp"` uses a reference QP; `auto` may use the QP fallback for at most 2000 rows.
`raking`	Entropic distance with positive starting weights; uses the full derivative including variable `q`.
`logit`	Finite vector bounds strictly enclosing positive starting weights; stable logistic formulas.

Raking and logit do not accept `metric_scale`. Their multiplier algorithms seek interior solutions; boundary-only targets may require `truncated` instead. Nonconvergence is an error, not proof of infeasibility.

```
calibration_distance(bounded_fit$weights, d, method = "truncated",
                     lower = 1, upper = c(rep(9, 9), 10))
#> [1] 0.54
calibration_distance_from_factors(bounded_fit$g, d, method = "truncated",
                                  lower = 1, upper = c(rep(9, 9), 10))
#> [1] 0.54
linear_multipliers(X, d, totals)
#> group_A group_B     age
#>   -0.34   -0.76    0.10
calibrate(X, d, totals, description = TRUE)  # Short wrapper with the same fit object
#> CalTools: linear calibration ( linear )
#> Status: optimal  | rank: 3  | max scaled residual: 2.73e-16
#> Weight range: 3.8 to 6.2
#> CalTools: linear calibration ( linear )
#> Status: optimal  | rank: 3  | max scaled residual: 2.73e-16
#> Weight range: 3.8 to 6.2
```

Fit objects contain weights, factors `g`, multipliers `lambda`, objective, residuals, numerical rank and solver details. Distances from different methods are not comparable measures of statistical performance. Multipliers need not be unique when columns are dependent.

0.3 2. Negative totals are supported

Negative totals do **not** require negative initial weights. For example, an auxiliary variable can take negative values while all survey weights are positive. The residual check is

$$\max_j \frac{|(X^\top w - t)_j|}{\max(1, |t_j|)} \leq \texttt{tol}.$$

It uses the absolute magnitude of each total and remains defined at zero.

```
X_signed <- cbind(N = rep(1, 5), signed = -1:-5,
                  contrast = c(-2, -1, 0, 1, 2))
target_weights <- c(1, 2, 3, 2, 1)
signed_totals <- setNames(as.numeric(crossprod(X_signed, target_weights)),
                          colnames(X_signed))
signed_totals
#>        N   signed contrast
#>        9      -27        0
signed_fit <- calibrate_weights(X_signed, rep(2, 5), signed_totals,
  q = c(.4, .8, 1, 1.3, 2), method = "truncated",
  lower = c(.1, .2, .3, .4, .5), upper = c(4, 5, 6, 5, 4))
signed_fit$residuals
#>             N        signed      contrast
#>  3.552714e-15 -1.065814e-14  0.000000e+00
```

The test suite also flips the signs of a column and its total together, and checks that the fitted weights do not change. An impossible negative total with a nonnegative column and nonnegative weight bounds is correctly rejected.

0.4 3. Negative or zero starting weights require an explicit metric

This is a different issue from negative totals. Using a negative starting weight as a quadratic denominator can destroy convexity. Select a positive scale explicitly.

```
X_small <- cbind(N = 1, x = 1:4)
starting_weights <- c(-1, 0, 2, 3)
fit <- calibrate_weights(X_small, starting_weights, c(N = 8, x = 22),
  method = "truncated", lower = 0, upper = 5, metric_scale = 1)
fit$weights
#> [1] 1.5 1.5 2.5 2.5
fit$g  # NA where the starting weight is zero
#> [1] -1.5000000         NA  1.2500000  0.8333333
```

Use `$weights` when a starting weight is zero. If factor bounds are converted for negative `d`, their order reverses: use `pmin(lower_factor*d,upper_factor*d)` and `pmax(lower_factor*d,upper_factor*d)`. Positive factor bounds preserve the sign of `d`; use final-weight bounds when negative weights must become positive.

0.5 4. Find and explain incompatible totals

The system is consistent precisely when appending $t^\top$ to X does not increase its row-space rank. Equivalently, every relation $Xc = 0$ must satisfy $c^\top t = 0$. The numerical implementation scales columns, computes an SVD and checks the new row against the resulting row space.

```
X_conflict <- cbind(group_A = c(1, 0), group_B = c(0, 1), N = c(1, 1))
conflicting_totals <- c(group_A = 4, group_B = 6, N = 11)
analyze_inconsistency(X_conflict, conflicting_totals)
#> [1] TRUE
report <- analyze_inconsistency(X_conflict, conflicting_totals, details = TRUE)
report
#> Calibration totals: incompatible
#> Rank of X: 2  | rank after appending totals: 3
#>                   relation discrepancy tolerance
#>  group_A + group_B - N = 0          -1   1.2e-07
report$incompatibilities
#>                             relation discrepancy tolerance sample_residual
#> relation_1 group_A + group_B - N = 0          -1   1.2e-07    2.664535e-15
#>            violated      margins
#> relation_1     TRUE group_A,....
report$coefficients
#>            group_A group_B  N
#> relation_1       1       1 -1
```

`details=FALSE` preserves a simple logical result. `details=TRUE` reports:

- `rank_X` and `rank_augmented`, using the stated numerical tolerances;
- `relations` and their numerical coefficient matrix;
- `incompatibilities`: violated relations, discrepancies, thresholds and involved margins;
- `projected_totals` and `residuals`, for diagnosis only.

The example exposes the conflict between `group_A + group_B` and `N`. It does not decide which source total is incorrect. The relation set is not promised to be minimal or independent. Input totals are never changed.

`rank_tol` determines numerical rank; `tol` determines admissible discrepancies in the original total units. Near numerical dependence, both settings matter.

```
X_redundant <- cbind(X, N = X[, "group_A"] + X[, "group_B"], empty = 0)
analyze_collinearity(X_redundant, simple = TRUE)
#> $multicollinear
#> [1] TRUE
#>
#> $relations
#>      group_A group_B age N empty
#> [1,]      -1      -1   0 1     0
#> [2,]       0       0   0 0     1
#>
#> $indices
#> [1] 4 5
#>
#> $rank
#> [1] 3
analyze_zero_sample(X_redundant)
#> $indices
#> empty
#>     5
#>
#> $names
#> [1] "empty"
```

Redundancy alone is harmless if totals satisfy the same relations. A zero column with a nonzero total is inconsistent. The zero-sample diagnostic uses an absolute threshold; it never removes constraints.

0.5.1 Nearest algebraically consistent totals

`analyze_inconsistency(..., details = TRUE)$projected_totals` is a diagnostic computed after **column normalization**. To get the genuinely nearest vector in the **original total units**, use `closest_consistent_totals()`:

```
X_conflict <- cbind(group_A = c(1, 0), group_B = c(0, 1), N = c(1, 1))
conflicting_totals <- c(group_A = 4, group_B = 6, N = 11)
closest_consistent_totals(X_conflict, conflicting_totals)
#>   group_A   group_B         N
#>  4.333333  6.333333 10.666667
# group_A = 4.333333, group_B = 6.333333, N = 10.666667
closest_consistent_totals(X_conflict, conflicting_totals,
                          scales = c(1, 1, 10), details = TRUE)
#> Closest consistent totals | rank: 2 | initially consistent: FALSE
#> Scaled Euclidean distance: 0.09901475
#>         original  adjusted   adjustment
#> group_A        4  4.009804  0.009803922
#> group_B        6  6.009804  0.009803922
#> N             11 10.019608 -0.980392157
```

It minimizes `sum(((t_star - totals)/scales)^2)/2`, subject to the existence of **some** (possibly unbounded or negative) `w` with `crossprod(X,w)=t_star`. The optional `scales` control which totals are trusted most; larger values permit larger changes. **This does not ensure bounded-weight feasibility.** Check the adjusted vector again with `calibration_preflight()` if needed.

0.6 5. Initial screening and bounded feasibility

```
calibration_preflight(X_conflict, conflicting_totals, lower = 0, upper = 10)
#> CalTools preflight: inconsistent_totals
#> Calibration totals: incompatible
#> Rank of X: 2  | rank after appending totals: 3
#>                    relation discrepancy tolerance
#>  group_A + group_B - N = 0          -1   1.2e-07
consistent_totals <- c(group_A = 4, group_B = 6, N = 10)
calibration_preflight(X_conflict, consistent_totals, lower = 0, upper = 3)
#> CalTools preflight: bound_infeasible
#> Calibration totals: consistent
#> Rank of X: 2  | rank after appending totals: 2
#> Equalities are consistent, but no weights satisfy the supplied bounds.
calibration_preflight(X_conflict, consistent_totals, lower = 0, upper = 10)
#> CalTools preflight: feasible
#> Calibration totals: consistent
#> Rank of X: 2  | rank after appending totals: 2
```

The statuses separate `inconsistent_totals`, `bound_infeasible`, `feasible` and `numerical_failure`. Moving bounds cannot repair incompatible equalities. Preflight and bounded feasibility require finite bounds.

```
check <- calibration_feasibility(X, totals, lower = 1, upper = 10)
check[c("feasible", "status", "solver", "max_scaled_residual")]
#> $feasible
#> [1] TRUE
#>
#> $status
#> [1] "feasible"
#>
#> $solver
#> [1] "dual-witness"
#>
#> $max_scaled_residual
#> [1] 4.440892e-16
calibration_feasibility(X, totals, 1, 10, method = "lp")$feasible
#> [1] TRUE
is_feasible(X, totals, lower = 1, upper = 10)
#> [1] TRUE
is_feasible_fast(X, totals, lower = 1, upper = 10)
#> [1] TRUE
is_feasible_lp(X, totals, lower = 1, upper = 10)
#> [1] TRUE
```

`TRUE` includes a checked witness. `FALSE` comes from an inconsistent equality system, a box-support certificate, or the LP solver's infeasible status. `NA` means inconclusive. Boolean wrappers raise an error on an inconclusive result. The LP uses $w = L + z$, with $z \geq 0$, and sparse constraint triplets. `is_feasible_fast` shares the validated default implementation with `is_feasible`.

0.7 6. Projection algorithms: scope and limitations

For the affine set $C = \{w : X^\top w = t\}$ and the box $B = [L, U]$, ordinary alternating projections iterate $w^{k+1} = P_B(P_C(w^k))$. They allow bound activity to change.

```
projection_check <- calibration_feasibility(X, totals, 1, 10,
  method = "projection", max_iter = 5000, fallback = FALSE)
projection_check[c("feasible", "status", "solver", "max_scaled_residual")]
#> $feasible
#> [1] TRUE
#>
#> $status
#> [1] "feasible"
#>
#> $solver
#> [1] "alternating-projections"
#>
#> $max_scaled_residual
#> [1] 5.921189e-16
```

The affine projection reuses an orthonormal basis. Its per-iteration cost is $O(nr)$ after reduction, where r is the constraint rank. With `fallback=FALSE`, iteration exhaustion returns `NA`, not `FALSE`; with `fallback=TRUE`, it runs the LP. `method="auto"` remains the quadratic dual with LP fallback, not ordinary alternating projections. Ordinary alternating projections are used here only to find a feasible witness, not to claim the closest calibrated weights.

The original permanent-clipping procedure is different: it could fix an incorrect face and report infeasibility. A four-unit counterexample and the full explanation are in MATHEMATICS.md.

Projection-based feasibility and best approximation are established subjects: Bauschke and Borwein (1996), and Boyle and Dykstra (1986), provide key references. Devaud and Tille (2019, Section 3.2.2) discuss closely related reduced Newton calibration, singularity at tight bounds and a more expensive general alternative. This supports the computational motivation, not the correctness of irreversible clipping. Farkas' lemma is a theorem of alternatives, not a competing runtime algorithm. Comparisons depend on the particular LP formulation and solver. See PROJECTIONS_AND_LITERATURE.md.

0.8 7. Relax bounds along a declared direction

`relax_bounds` minimizes $\rho \geq 0$ subject to calibration and $L_i - \rho a_i \leq w_i \leq U_i + \rho b_i$. The rates are nonnegative scalar or vector arguments. This is the minimal expansion along that direction, not the nearest box under every possible metric.

```
relax_bounds(matrix(1, 3, 1), 9, 0, 2, lower_rate = 0, upper_rate = 1)
#> $status
#> [1] "optimal"
#>
#> $rho
#> [1] 1
#>
#> $lower
#> [1] 0 0 0
#>
#> $upper
#> [1] 3 3 3
```

```
#>
#> $weights
#> [1] 3 3 3
analyze_feasibility_bounds(X, totals, d, type = "g-weight", method = "upper",
  bounds = c(.9, 1.1), search_limits = 2, step = .1)
#> $bounds
#> [1] 0.9 1.6
#>
#> $lower
#>  [1] 4.5 4.5 4.5 4.5 4.5 4.5 4.5 4.5 4.5 4.5
#>
#> $upper
#>  [1] 8 8 8 8 8 8 8 8 8 8
#>
#> $weights
#>  [1] 4.5 4.5 4.5 4.5 6.0 4.5 4.5 4.5 4.5 8.0
#>
#> $status
#> [1] 0
#>
#> $grid_index
#> [1] 6
analyze_feasibility_bounds(matrix(1, 2, 1), 1, c(10, 20),
  type = "w-weight", method = "lower", bounds = c(1, 2), search_limits = 0, step = .25)
#> $bounds
#> [1] 0.5 2.0
#>
#> $lower
#> [1] 0.5 0.5
#>
#> $upper
#> [1] 2 2
#>
#> $weights
#> [1] 0.5 0.5
#>
#> $status
#> [1] 0
#>
#> $grid_index
#> [1] 3
```

The grid interface uses binary search on nested boxes. `both` expands the two endpoints simultaneously until their respective search limits. It is not an exhaustive search over all endpoint pairs. The final endpoint is always included. `g-weight` endpoints multiply starting weights; `w-weight` endpoints are absolute.

0.9 8. Modify margins explicitly

```
merged <- merge_margins(X, totals, c("group_A", "group_B"), "N")
merged$totals
#> age   N
```

```
#> 290  50
drop_margins(X, totals, "age")$totals
#> group_A group_B
#>      24      26
check_modified_feasibility(X, totals, d, lower = 4.5, upper = 5.5,
  margin_names = c("group_A", "group_B"), action = "join", merged_name = "N",
  bounds = c(.9, 1.1), search_limits = 2, step = .1)
#> $X
#>       age N
#>  [1,]   1 1
#>  [2,]   2 1
#>  [3,]   3 1
#>  [4,]   4 1
#>  [5,]   5 1
#>  [6,]   6 1
#>  [7,]   7 1
#>  [8,]   8 1
#>  [9,]   9 1
#> [10,]  10 1
#>
#> $totals
#> age   N
#> 290  50
#>
#> $bounds
#> [1] 0.9 1.2
#>
#> $lower
#>  [1] 4.5 4.5 4.5 4.5 4.5 4.5 4.5 4.5 4.5 4.5
#>
#> $upper
#>  [1] 6 6 6 6 6 6 6 6 6 6
#>
#> $weights
#>  [1] 4.5 4.5 4.5 4.5 4.6 4.9 5.2 5.5 5.8 6.0
#>
#> $status
#> [1] 0
#>
#> $grid_index
#> [1] 2
check_sequential_feasibility(X, totals, d, lower = 4.5, upper = 5.5,
  margin_names_list = list(c("group_A", "group_B"), "age"),
  actions = c("join", "suppress"), merged_names = list("N", NULL),
  bounds = c(.9, 1.1), search_limits = 2, step = .1)
#> $X
#>       age N
#>  [1,]   1 1
#>  [2,]   2 1
#>  [3,]   3 1
#>  [4,]   4 1
#>  [5,]   5 1
#>  [6,]   6 1
```

```
#>  [7,]   7 1
#>  [8,]   8 1
#>  [9,]   9 1
#> [10,]  10 1
#>
#> $totals
#> age   N
#> 290  50
#>
#> $bounds
#> [1] 0.9 1.2
#>
#> $lower
#>  [1] 4.5 4.5 4.5 4.5 4.5 4.5 4.5 4.5 4.5 4.5
#>
#> $upper
#>  [1] 6 6 6 6 6 6 6 6 6 6
#>
#> $weights
#>  [1] 4.5 4.5 4.5 4.5 4.6 4.9 5.2 5.5 5.8 6.0
#>
#> $status
#> [1] 0
#>
#> $grid_index
#> [1] 2
#>
#> $iter
#> [1] 1
#>
#> $history
#> $history[[1]]
#> $history[[1]]$action
#> [1] "join"
#>
#> $history[[1]]$variables
#> [1] "group_A" "group_B"
```

Merging sums columns and totals; dropping removes equations. Both weaken the constraints and do not guarantee the original separate totals. Names and their order must match. Zero-column matrices are supported. The sequential function returns the first feasible result, or the last attempted problem with its history. These checks try the supplied vector bounds first; their common-bound grid can replace that box. Use directional relaxation instead if individual limits must retain their substantive meaning.

0.10 9. Automatic and interactive workflows

```
margin_groups <- list(c("group_A", "group_B"))
recommend_margin_merges(X, d, totals, margin_groups = margin_groups,
                        ordered = FALSE, criterion = "sample")
#> [[1]]
#> [1] "group_A" "group_B"
```

```
recommend_margin_merges(X, d, totals, margin_groups = margin_groups,
                        ordered = FALSE, criterion = "lambda")
#> [[1]]
#> [1] "group_B" "group_A"
adjacent_elements(c("young", "middle", "older"), 2)
#> [1] "young" "older"
```

Recommendations are heuristics. Multiplier scores are scaled by column norms but still depend on a representation when columns are dependent. Groups must be disjoint. `ordered=TRUE` permits only adjacent pair merges within a linear order.

```
X_search <- diag(3)
colnames(X_search) <- c("A", "B", "C")
search_totals <- c(A = 0, B = 0, C = 6)
candidates <- automatic_calibration(X_search, search_totals, rep(1, 3),
  lower = 1, upper = 3, bounds = c(1, 3), search_limits = c(1, 3),
  margin_groups = list(c("A", "B", "C")), ordered = TRUE,
  max_depth = 2, max_solutions = 3)
candidates
#> CalTools search: 1 candidate(s); exhausted
#> CalTools preflight: bound_infeasible
#> Calibration totals: consistent
#> Rank of X: 3  | rank after appending totals: 3
#> Equalities are consistent, but no weights satisfy the supplied bounds.
#> Candidate 1 | merges: 2 | objective: 1.5
attr(candidates, "preflight")
#> CalTools preflight: bound_infeasible
#> Calibration totals: consistent
#> Rank of X: 3  | rank after appending totals: 3
#> Equalities are consistent, but no weights satisfy the supplied bounds.
candidates[[1]]$history
#> [[1]]
#> [[1]]$action
#> [1] "join"
#>
#> [[1]]$variables
#> [1] "A" "B"
#>
#> [[1]]$name
#> [1] "A+B"
#>
#>
#> [[2]]
#> [[2]]$action
#> [1] "join"
#>
#> [[2]]$variables
#> [1] "A+B" "C"
#>
#> [[2]]$name
#> [1] "A+B+C"
candidates[[1]]$residuals_original
#>  A  B  C
```

```
#>  2  2 -4
```

The breadth-first search updates groups and deduplicates equivalent partitions. It stops at the requested solution, depth or node limits. Every candidate includes the actual matrix, totals, bounds, weights, history and original residuals. It never deletes a nonzero population total silently.

```
X_inconsistent <- cbind(A = c(1, 0), B = c(2, 0))
bad_totals <- c(A = 2, B = 5)
stopped <- automatic_calibration(X_inconsistent, bad_totals, c(1, 1),
  lower = 0, upper = 10, margin_groups = list(c("A", "B")), ordered = TRUE)
stopped  # No bound search or fitting: the original equalities conflict.
#> CalTools search: 0 candidate(s); inconsistent_totals
#> CalTools preflight: inconsistent_totals
#> Calibration totals: incompatible
#> Rank of X: 1  | rank after appending totals: 2
#>         relation discrepancy tolerance
#>  A - 0.5 * B = 0        -0.5     4e-06
# Explicitly permit changing the equalities through aggregation:
merged_candidates <- automatic_calibration(X_inconsistent, bad_totals, c(1, 1),
  lower = 0, upper = 10, margin_groups = list(c("A", "B")), ordered = TRUE,
  on_inconsistency = "merge")
merged_candidates[[1]]$residuals_original
#>          A          B
#>  0.3333333 -0.3333333
```

`on_inconsistency="stop"` is the default. It returns an empty search with the initial diagnostic, so you can inspect the conflict before editing anything. `"merge"` allows weakened problems, not a claim that the original conflict vanished. At every search node, inconsistent equalities skip the bound-relaxation stage.

```
selected <- interactive_calibration(X_search, search_totals, rep(1, 3),
  lower = 1, upper = 3, margin_groups = list(c("A", "B", "C")), ordered = TRUE,
  bounds = c(1, 3), search_limits = c(1, 3),
  choose = function(candidates) 1L)
selected$fit
#> CalTools: truncated calibration ( dual )
#> Status: optimal  | rank: 1  | max scaled residual: 2.96e-16
#> Weight range: 2 to 2
selected$preflight$status
#> [1] "bound_infeasible"
```

In an interactive R console, omit `choose` to see the initial diagnostic and choose a candidate; zero cancels. In scripts provide `choose`. No candidates returns an empty search with diagnostics; cancellation returns `NULL`.

0.11 10. Random inclusion probabilities

```
set.seed(12)
probabilities <- random_inclusion_probabilities(N = 100, n = 10)
c(sum = sum(probabilities), minimum = min(probabilities), maximum = max(probabilities))
#>          sum      minimum      maximum
#> 10.000000000  0.006041989  0.418737381
```

The generator normalizes exponential sizes with certainty units. It returns values in $[0, 1]$ summing to `n`, including the endpoints `n=0` and `n=N`. It is not uniform on the fixed-sum polytope and does not reproduce the old beta-rejection distribution.

0.11.1 Reproducible random calibration stress tests

The generator constructs categorical partitions (complete sets of dummy variables), binary, signed and positive numeric variables, zero-sample margins and exactly dependent columns. It returns the `X`, `totals`, `d`, `q`, vector bounds and `margin_groups` accepted directly by the current public API.

```
feasible_case <- generate_calibration_problem(
  n = 30, categorical_groups = c(5, 4), n_binary = 2, n_continuous = 1,
  n_zero = 1, n_derived = 1, difficulty = .25, seed = 42)
calibration_preflight(feasible_case$X, feasible_case$totals,
                      feasible_case$lower, feasible_case$upper)$status
# "feasible"

hard_case <- generate_calibration_problem(
  n = 30, categorical_groups = c(5, 4), n_binary = 2,
  difficulty = .8, stress_pattern = "local", seed = 42)
calibration_preflight(hard_case$X, hard_case$totals,
                      hard_case$lower, hard_case$upper)$status
# "bound_infeasible"

# Use the same generated object in the new automatic workflow.
candidates <- automatic_calibration(hard_case$X, hard_case$totals,
  hard_case$d, q = hard_case$q,
  lower = hard_case$lower, upper = hard_case$upper,
  bounds = hard_case$bounds, method = "upper", search_limits = 3,
  step = .1, margin_groups = hard_case$margin_groups,
  ordered = hard_case$ordered, max_solutions = 3, max_depth = 2)

bad_case <- generate_calibration_problem(
  n = 30, categorical_groups = c(5, 4), inconsistency = "linear", seed = 42)
analyze_inconsistency(bad_case$X, bad_case$totals)
# TRUE
closest_consistent_totals(bad_case$X, bad_case$totals)
```

Operational difficulty: for `difficulty <= 0.5`, a known solution meets all initial bounds; for `difficulty > 0.5`, the totals violate the box support of at least one margin and require relaxing or weakening constraints. With `stress_pattern="local"`, this targets a low-frequency category to exercise aggregation strategies; with `"global"`, it stresses all observations and is better for bound-expansion benchmarks. `max_excess` sets the maximum extra excursion. The parameter does **not** prescribe a particular number of search nodes or margin merges. `inconsistency="linear"` or `"zero"` independently creates contradictory equations; these cannot be fixed by bounds alone.

Legacy-style fields `Xs`, `total`, `L_bounds`, `U_bounds`, `mar_2_join` and `hierarchy` are available on the generated object, but the **package function names** follow the 0.2.x migration (`automatic_calibration`, `calibration_preflight`, `analyze_collinearity`, etc.). For repeated runs see `inst/benchmarks/random-problems.R`.

0.12 11. Reproducibility and limits

Every public function has roxygen2 documentation and runnable help examples. Tests cover analytic solutions, an independent QP reference, negative and zero totals, sign invariance, rank diagnostics, preflight

policies and search depth.

```
# From the directory containing the package project:
roxygen2::roxygenise("CalTools")
testthat::test_local("CalTools")
# After installing the current package:
rmarkdown::render("CalTools/README.Rmd")
# In a terminal:
# R CMD build CalTools
# R CMD check --no-manual CalTools_0.2.1.tar.gz
```

The numerical core is dense. Initial reduction costs about $O(np^2)$ for $n \geq p$. A dual Newton iteration costs $O(nr^2 + r^3)$; an alternating-projection iteration costs $O(nr)$. Detailed relation reporting adds a $p \times p$ projector. The QP fallback has quadratic memory in `n`, hence its 2000-row limit. Finite iteration budgets are not universal convergence guarantees. Validation uses synthetic data; no performance or statistical-efficiency claim is made for unseen survey data.

Additional documentation:

- Mathematical derivations
- Projection methods and literature
- Audit and API migration
- Validation record
- Benchmarks in `inst/benchmarks/` and generated help in `man/`.